AI-Based Exam Proctoring System

Frontend Developer Handover & Implementation Specification
React web application – Instructor Dashboard + student sign-up (Arabic / English)

Document date	2026-10-23 (revision 2: owner scope decisions applied)
Backend verified at	Git commit `f0c11e3093b5e8bc2077aed83ea0351b9eee3729` (branch `main`, local, not pushed)
Backend scope covered	Phase 0, Phase 1 (authentication, admin accounts) and Phase 2 Steps 2.1–2.4 (exams, questions, lifecycle)
Source of truth	Backend source code in `src/`; approved plans and open decisions in `architecture-proposal.md` (Revision 3.10)
Audience	The frontend developer (working with Claude Code) and the project team

Labels used in this document
IMPLEMENTED exists in the backend source and is covered by passing automated tests · RECOMMENDATION frontend advice, not a backend contract · PLANNED documented in the architecture but not built · BLOCKED waits for an unresolved architecture decision · UNVERIFIED could not be confirmed from the repository

Contents

0. How this document was produced
A. Project overview
B. Exact frontend scope
C. Complete backend API reference
D. Business logic and UI behaviour
E. Authentication and security
F. Frontend-to-backend integration
G. Recommended React project structure
H. UI/UX and validation specification
I. Error handling matrix
J. Implementation plan
K. Testing and acceptance checklist
L. Ready-to-use Claude Code prompt

0. How this document was produced

Every route, field, status code, message and rule below was read from the backend source code at commit `f0c11e3`:

- Controllers: `src/ExamProctoring.Api/Controllers/` (`AuthController`, `AdminAccountsController`, `ExamsController`, `QuestionsController`).
- DTOs, validators and services: `src/ExamProctoring.Application/` (`Auth/`, `Accounts/`, `Exams/`, `Common/ErrorCodes.cs`).
- Entities and rules: `src/ExamProctoring.Domain/`; EF Core configuration and migrations: `src/ExamProctoring.Infrastructure/`.
- Error handling: `src/ExamProctoring.Api/Errors/`, `Middleware/GlobalExceptionHandler.cs`, `Authorization/`.
- Configuration: `appsettings.json`, `appsettings.Development.json`, `Properties/launchSettings.json`.
- Tests: `tests/ExamProctoring.IntegrationTests` (587 tests) and `tests/ExamProctoring.UnitTests` (368 tests). The latest full run (2026-10-23, at `f0c11e3`) passed 955 of 955; it was not re-run for this document.

> **Limitations of this review.**
>
> - The original requirements PDF (`Requirments/Analysis Pahse Updated.pdf`) could not be rendered in this environment; the requirements summary relies on §1 of `architecture-proposal.md`, which was written from that PDF.
> - Behaviour is taken from source code and the integration tests. On 2026-10-23 the owner's local API (launch profile `http`) was also probed live: `GET /health` → 200 and `POST /api/auth/login` with an empty body → 400 `validation_failed`. No other live calls were made for this document.
> - `architecture-proposal.md` Revision 3.10 (decisions D25–D36) is **not committed**; the committed copy is Revision 3.9, which predates the exam decisions. The repository has no Git remote yet; this document, `frontend/` and Revision 3.10 must be committed and pushed to the GitHub repository (owner's step) before a developer can receive them.
> - The `frontend/` draft (untracked) has never been installed, type-checked, built or tested, because Node.js was not installed on the machine that wrote it. The owner's decision is to **reuse** it (Section G.2), verifying it first.

Owner decisions applied in revision 2 (2026-10-23)

1. **Scope:** the React app is the **Instructor Dashboard**.
2. **Admin screens are excluded** from React; admin functions belong to the Flutter app. The admin API is documented for reference only (C.6).
3. **Student sign-up is included**, using only the existing `POST /api/auth/register` contract (C.3, D.10). After sign-up the student sees a success message with a sign-in link; there is no automatic sign-in.
4. **The existing `frontend/` draft is reused**, not replaced (G.2).
5. **Arabic and English** with a language toggle; Arabic is right-to-left, English left-to-right; **Arabic is the default** on a first visit (H.9).
6. **Delivery** through the project's GitHub repository. Nothing is committed or pushed without the owner's explicit approval.

A. Project overview

A.1 Purpose

The AI-Based Exam Proctoring System provides a controlled environment for university exams: identity verification, AI proctoring, randomized exams, automated grading where possible, AI question generation, integrity scores and collusion reports. Actors are Instructor, Student, Administrator and the AI proctoring module.

A.2 Backend technology (verified)

Item	Verified value
Framework	ASP.NET Core Web API, .NET 10 (SDK 10.0.402 pinned in `global.json`); controllers with `[ApiController]`
Data	SQL Server with EF Core 10 (compatibility level 160); migrations applied manually, never at startup
Identity	ASP.NET Core Identity (`AddIdentityCore`, no cookies) + JWT bearer tokens (HS256) + rotating refresh tokens stored hashed in the database
Validation	FluentValidation in the Application layer
Errors	RFC 9457 ProblemDetails (`application/problem+json`) with a stable `code` extension
API documentation	OpenAPI document `/openapi/v1.json` and Swagger UI `/swagger`, Development environment only; no JWT "Authorize" button yet
Projects	`Domain`, `Application`, `Infrastructure`, `Api`, `UnitTests`, `IntegrationTests` in `ExamProctoring.slnx`

A.3 The two client applications

Client	Role according to the architecture (§3.1)	This handover
Web exam app (website)	Student only: login, face enrollment, exams intended for the student, identity check, camera/microphone checks, fullscreen lockdown, randomized exam, autosave, submit, in-browser violation detectors.	The student pages are **BLOCKED** / **PLANNED**: no student exam endpoint exists.
Flutter mobile app	Student sign-up, login for all roles, instructor exam/schedule/question management, results, reports; admin account functions.	Out of scope (not built here).

> **Recorded deviation from the architecture.** Architecture §3.1 assigns instructor exam management *and* student sign-up to the Flutter app and makes the website student-only. The owner decided (2026-10-23) that the *React* app is the Instructor Dashboard and also offers student sign-up. **Impact:** none on the API (these endpoints work for any HTTP client). The CORS setting is still named for the web exam app (`Cors:WebExamOrigins`), and `architecture-proposal.md` has not been updated to record this decision. Follow this document, not §3.1, for the React scope.

A.4 Responsibilities

Backend (authoritative)	Frontend

Authentication, account status, roles, forced password change, ownership checks on every request; all validation and business rules; state transitions; time checks against the server clock; final error decisions.	Collect input, give early validation feedback, call the API, show results and server errors, keep the UI consistent with the server's answers, never assume an action is allowed just because the UI showed a button.

A.5 Implementation status

Area	Status	Notes
Authentication (login, refresh, logout, me, change password, student sign-up)	IMPLEMENTED	Phase 1
Admin account management	IMPLEMENTED	Admin role only; architecture assigns it to Flutter
Exam management (Step 2.2)	IMPLEMENTED	commit `5b515bc`
Question management (Step 2.3)	IMPLEMENTED	commit `20259a7`
Publish / unpublish / cancel (Step 2.4)	IMPLEMENTED	commit `f0c11e3`; the "no attempt exists" unpublish check is deferred to Phase 4
Student exam list/details (Step 2.5)	BLOCKED	Open decision §2.4: how students are assigned to exams
Attempts, identity verification, randomization, submission, grading, results	PLANNED	Phase 4
Lecture upload, AI question generation	PLANNED	Phase 3
Proctoring events, warnings, termination	PLANNED	Phase 5 (the exam's `warningLimit` is stored but unused)
Integrity score, collusion report	PLANNED	Phase 6
Password recovery by email	PLANNED	Decision D17; details open

B. Exact frontend scope

B.1 Scope categories

Category	Features
IMPLEMENTED build now	Instructor sign-in; forced password change after a temporary password; sign-out; session refresh; exam list with status filter and pagination; create, view, edit and delete (Draft) exams; question list, create, replace, delete (MCQ and Essay); publish, unpublish, cancel; **student sign-up** (`POST /api/auth/register`, D.10); **Arabic/English with RTL/LTR** and a language toggle (H.9).
Implemented but excluded from React (owner decision)	The admin account endpoints (`/api/admin/*`, C.6). Admin screens belong to the Flutter app. Do not build admin pages. For local testing, an instructor account is created by calling the admin API directly (F.6).
PLANNED not implemented	Exam taking, identity verification, webcam/microphone monitoring, warnings, results, grading, AI question generation, integrity and collusion reports, password recovery.
BLOCKED	Student "my exams" list and exam info (`GET me/exams`) – no endpoint exists; depends on architecture §2.4.

B.2 Pages

Page / route (recommended)	Access	Purpose, data and components	API calls
Sign in `/login`	Anonymous	Email + password form. On success: if `mustChangePassword` → change-password page; else → exam list (or the page the user tried to open). Errors shown inline (Section I).	`POST /api/auth/login`
Sign up `/register`	Anonymous	Student sign-up form: full name, email, password, confirm password (exactly the four backend fields). On 201 show the success panel from D.10 (no tokens are returned; no automatic sign-in). Link to it from the sign-in page ("Create a student account").	`POST /api/auth/register`
Change password `/change-password`	Any signed-in account	Current (temporary) password, new password, repeat new password (repeat is frontend-only). Mandatory while `mustChangePassword` is true; every other API call answers 403 `password_change_required` until done.	`POST /api/auth/change-password`
Layout (all pages below)	Instructor	Header: app name, links "Exams" and "New exam", user's full name, language toggle (العربية / English), Sign out. The toggle is also shown on the sign-in, sign-up and change-password pages.	`POST /api/auth/logout`
Exam list `/exams`	Instructor	Table (cards on phones): title (+ description excerpt), start, duration, status badge, phase badge. Status filter (All/Draft/Published/Cancelled), page size, previous/next pagination with "x–y of total". Empty state with "Create your first exam".	`GET /api/exams`
Create exam `/exams/new`	Instructor	Exam form (title, description, start date/time with the user's time-zone hint, duration). On success go to the new exam's details page.	`POST /api/exams`

Exam details `/exams/:id`	Instructor (owner)	Title, status and phase badges, description, start, end, duration, published time, warning limit (read-only). Action bar: Edit, Publish, Unpublish, Cancel exam, Delete (per D.4). Questions panel below.	`GET /api/exams/{id}`, lifecycle POSTs, `DELETE /api/exams/{id}`
Edit exam `/exams/:id/edit`	Instructor (owner)	Same form, prefilled. Only reachable while the exam is Draft/Published and has not started; otherwise show an explanation.	`GET` + `PUT /api/exams/{id}`
Questions panel (inside details)	Instructor (owner)	Ordered list (Q1, Q2 …) with type, marks, text, options with the correct one marked, model answer; total marks. Add / Edit / Delete when editable. Inline question editor (D.5, H.6).	`GET` / `POST /api/exams/{id}/questions`, `PUT` / `DELETE /api/questions/{id}`
Not found / no access	Any	Unknown route; non-instructor account ("Instructor access only" with Sign out). A signed-in **Student** sees the same screen with the D.10 note that student exam features are not available yet.	–

B.3 Main user flows

1. **First sign-in of a new instructor:** an admin created the account (via the Flutter app or `POST /api/admin/instructors`) and gave the instructor a temporary password → sign in → forced change-password page → exam list.
2. **Prepare an exam:** New exam → details page → add questions → Publish (server validates the whole exam) → status Published.
3. **Change before the start:** edit details or questions of a Draft or Published exam while it has not started; unpublish to return to Draft; or cancel (irreversible).
4. **After the start time:** the exam and its questions are read-only; a Draft can still be deleted.
5. **Student sign-up:** Sign up → 201 → success panel ("account created; student exam features are not available yet") with a link to sign in. Signing in works, but a Student has no pages in this app yet (§2.4 is open).

C. Complete backend API reference

C.1 Conventions (all endpoints)

- Base path `/api`. JSON bodies, **camelCase** property names. Enums are sent and returned as **strings** (`"Draft"`, `"Mcq"`). GUIDs are strings. Times are ISO 8601 strings; responses always use UTC (`+00:00`).
- Protected endpoints need `Authorization: Bearer <accessToken>`. Every protected endpoint also enforces: active account, unchanged security stamp, and the forced password change (except where noted).
- Errors: ProblemDetails with `status`, `detail`, a machine-readable `code` and, for validation errors, `errors` (field → messages). ASP.NET Core also adds `type`, `title` and usually `traceId`; don't rely on those.
- A role failure (e.g. a Student calling an instructor endpoint) is `403` *without* a `code`. A missing/invalid/expired token is `401` with `code: "token_invalid"` and header `WWW-Authenticate: Bearer`.
- Unknown ids, malformed ids and other instructors' data all answer `404` (existence is never revealed).
- Create endpoints return `201` with the body and **no** `Location` header.

C.2 Endpoint index

Method	Route	Access	Status
POST	`/api/auth/login`	Anonymous, 10/min/IP	IMPLEMENTED
POST	`/api/auth/refresh`	Anonymous, 30/min/IP	IMPLEMENTED
POST	`/api/auth/logout`	Signed in*	IMPLEMENTED
POST	`/api/auth/change-password`	Signed in*	IMPLEMENTED
GET	`/api/auth/me`	Signed in*	IMPLEMENTED
POST	`/api/auth/register`	Anonymous, 5/min/IP	IMPLEMENTED (student sign-up, in React scope)
GET	`/api/exams`	Instructor	IMPLEMENTED
POST	`/api/exams`	Instructor	IMPLEMENTED
GET	`/api/exams/{id}`	Instructor (owner)	IMPLEMENTED
PUT	`/api/exams/{id}`	Instructor (owner)	IMPLEMENTED
DELETE	`/api/exams/{id}`	Instructor (owner)	IMPLEMENTED
POST	`/api/exams/{id}/publish`	Instructor (owner)	IMPLEMENTED
POST	`/api/exams/{id}/unpublish`	Instructor (owner)	IMPLEMENTED
POST	`/api/exams/{id}/cancel`	Instructor (owner)	IMPLEMENTED
GET	`/api/exams/{examId}/questions`	Instructor (owner)	IMPLEMENTED
POST	`/api/exams/{examId}/questions`	Instructor (owner)	IMPLEMENTED
PUT	`/api/questions/{id}`	Instructor (owner)	IMPLEMENTED
DELETE	`/api/questions/{id}`	Instructor (owner)	IMPLEMENTED
GET	`/api/admin/accounts`, `/api/admin/accounts/{id}`	Admin	IMPLEMENTED (no React screens)

POST	`/api/admin/instructors`, `/api/admin/accounts/{id}/temporary-password`, `/deactivate`, `/reactivate`	Admin	**IMPLEMENTED** (no React screens)
GET	`/health`	Anonymous	**IMPLEMENTED** (not under `/api`)

* "Signed in": any role; allowed even while a password change is pending. Every other protected endpoint answers 403 `password_change_required` during a pending change.

> **Endpoints that do NOT exist** (do not call them): `GET /api/questions/{id}` (single question), any question reorder endpoint, `GET me/exams` and `GET me/exams/{id}`, any attempt, answer, result, grading, face-reference, lecture, generation, proctoring-event or report endpoint, password reset / forgot password, a settings endpoint that exposes `Exams:MaxMcqOptions`, and exam search by title.

C.3 Authentication endpoints

POST /api/auth/login

Request `LoginRequest`: `{ "email": string, "password": string }`. Body limit 16 KB. Validation: "Email is required.", "Password is required."

```
// 200 TokenResponse
{
  "tokenType": "Bearer",
  "accessToken": "<JWT>",
  "accessTokenExpiresAt": "2026-10-23T09:15:00+00:00",
  "refreshToken": "<opaque token>",
  "refreshTokenExpiresAt": "2026-10-30T09:00:00+00:00",
  "mustChangePassword": false,
  "user": { "id": "<guid>", "fullName": "Dr. Example", "email": "instructor@example.edu", "role":
"Instructor" }
}
```

Errors: 400 `validation_failed`; 401 `invalid_credentials` ("The email or password is incorrect."); 401 `account_locked` (after 5 failed attempts, 15 minutes); 403 `account_deactivated`; 403 `temporary_password_expired` ("The temporary password has expired. Ask an administrator for a new one."); 413 body too large; 429 rate limit. Role values: `Admin`, `Instructor`, `Student`.

POST /api/auth/refresh

Request `{ "refreshToken": string }`. 200 → a new `TokenResponse` (new access *and* refresh token; the old refresh token is used up). Errors: 400 `validation_failed`; 401 `token_invalid` ("The refresh token is invalid or has expired."); 401 `refresh_token_reused` (an already-rotated token was presented: the whole session family is revoked); 403 `account_deactivated` / `temporary_password_expired`; 429.

POST /api/auth/logout

Bearer token required. Request `{ "refreshToken": string }`. 204 (ends that session's token family; the current access token stays valid until it expires). Errors: 400, 401.

POST /api/auth/change-password

Bearer token required (allowed during a pending change). Request `{ "currentPassword": string, "newPassword": string }` (no confirmation field). 200 → new `TokenResponse`; every earlier session and access token of the account stops working. Errors: 400 `validation_failed` with field messages, e.g. `currentPassword`: "The current password is incorrect.", `newPassword`: "The new password must be different from the current password." or Identity policy messages; 401 `account_locked` after too many wrong current passwords; 401 `token_invalid`.

Password policy (Identity): at least 8 characters, an uppercase letter, a lowercase letter and a digit; no symbol required.

GET /api/auth/me

200 `{ "id", "fullName", "email", "role", "mustChangePassword" }`. 401 `token_invalid`.

POST /api/auth/register (student sign-up – in React scope)

Source: `AuthController.Register`, `RegisterStudentRequest`, `RegisterStudentRequestValidator`, `AccountFieldRules`, `EmailRules`, `AuthService.RegisterStudentAsync`. Anonymous; rate limit 5 per minute per IP. No body-size attribute (the 16 KB limit applies only to login, refresh, logout and change-password).

```
// Request RegisterStudentRequest – exactly these four fields
{ "fullName": "Sara Ahmed", "email": "sara@example.edu", "password": "Example1pass", "confirmPassword":
"Example1pass" }

// 201 RegisterStudentResponse – no tokens; the user is NOT signed in
{ "id": "<guid>", "fullName": "Sara Ahmed", "email": "sara@example.edu", "role": "Student" }
```

Field (error key)	Server rule and exact message
`fullName`	Required after trimming: "Full name is required."; at most 150 characters after trimming: "Full name must be at most 150 characters."; no control characters: "Full name must not contain control characters.". No other format rule; Arabic names are valid.
`email`	Required: "Email is required."; otherwise "Email is not a valid email address." unless, after trimming: 1–256 characters, no whitespace, control or invisible format (Unicode Cf) characters, exactly one `@` with text on both sides, and a domain that contains a dot, does not start or end with `.` or `-` and contains no `..`. Any email domain; non-English letters allowed.
`password`	Required: "Password is required."; then the Identity policy (at least 8 characters, an uppercase letter, a lowercase letter, a digit) with Identity's own messages.
`confirmPassword`	Must equal `password`: "Passwords do not match." Validated, never stored.

Other responses: 409 `duplicate_account` ("An account with this email already exists."; also for deactivated accounts and any role); 429 rate limit. Extra JSON properties (for example `role` or `universityId`) are ignored; the role is always Student and the account is Active immediately (no email verification).

> **University identification is not a blocker.** The Revision 3.3 open decision on identifying a student's university was **withdrawn in Revision 3.4** (D11, D12): any university and any email domain are accepted, accounts store no university data, and the sign-up form has exactly the four fields above. §2.5 keeps "university affiliation" only as a *future* question that "is not a registration requirement" and would never be added to the sign-up form. Do **not** add a university field, dropdown or ID.

C.4 Exam endpoints (Step 2.2 and 2.4)

Exam response `ExamResponse` (exact fields and order)

Field	Type	Meaning
`id`	string (GUID)	Exam id
`title`	string	1–200 characters
`description`	string | null	Up to 2000 characters
`startsAt`	string (date-time, UTC)	Start instant

`durationMinutes`	integer	1–360
`endsAt`	string (date-time, UTC)	Calculated: start + duration
`status`	`"Draft" | "Published" | "Cancelled"`	Lifecycle state
`phase`	`"Upcoming" | "Open" | "Closed"`	Calculated when read: before start / start ≤ now < end / from end on
`warningLimit`	integer	Copied from server configuration (20) at creation; read-only for instructors
`publishedAt`	string | null	Set on publish, cleared on unpublish, kept on cancel

GET /api/exams

The caller's own exams only. Query: `status` (optional; `Draft`, `Published`, `Cancelled`, any letter case; numbers rejected), `page` (≥ 1, default 1), `pageSize` (1–100, default 10). Sorted by `startsAt` newest first, then by id (fixed; no sort parameter). Invalid values → 400 (never adjusted): "Status must be Draft, Published or Cancelled.", "Page must be 1 or greater.", "Page size must be between 1 and 100."; non-numeric values → 400 `validation_failed` from model binding. A page beyond the end returns empty `items` with the real `totalCount`.

```
// 200 ExamListResponse
{ "items": [ /* ExamResponse */ ], "page": 1, "pageSize": 10, "totalCount": 12 }
```

POST /api/exams · PUT /api/exams/{id}

Request `ExamRequest` (same for create and full replacement):

```
{
  "title": "Midterm",
  "description": "Chapters 1-4",            // optional; null or blank = none
  "startsAt": "2026-11-01T11:00:00+02:00",  // offset REQUIRED (Z or ±hh:mm)
  "durationMinutes": 90
}
```

Create: **201** `ExamResponse` with `status: "Draft"`, `startsAt` converted to UTC (here `2026-11-01T09:00:00+00:00`), `warningLimit: 20`. Update: **200** `ExamResponse`; allowed only for Draft or Published exams before `startsAt`; status, warning limit and `publishedAt` never change. Extra fields (`status`, `warningLimit`, `publishedAt`, `id`, `instructorId`) are ignored.

Validation (400 `validation_failed`, field keys `title`, `description`, `startsAt`, `durationMinutes`): "Title is required.", "Title must be at most 200 characters.", "Description must be at most 2000 characters.", "Start time is required.", "Start time must be a date and time with a time-zone offset, for example 2026-11-01T09:00:00Z or 2026-11-01T11:00:00+02:00.", "Start time is too far in the future.", "Start time must be in the future." (create always; update only when the instant changes), "Duration is required.", "Duration must be between 1 and 360 minutes.". Lengths count after trimming.

Other errors: 404 `exam_not_found` (PUT); 409 `exam_not_editable` (PUT on a started or Cancelled exam: "The exam can no longer be changed in its current state."). Validation runs before the exam is looked up, so an invalid body for an unknown id still gives 400.

GET /api/exams/{id}

200 `ExamResponse`; 404 `exam_not_found` ("The exam was not found.").

DELETE /api/exams/{id}

Draft only (also after its start time); deletes its questions and options. 204 with an empty body. 409 `exam_not_editable` for Published or Cancelled; 404 `exam_not_found`.

POST /api/exams/{id}/publish · /unpublish · /cancel

No request body. Success: **200** with the updated `ExamResponse`. Refusal: 409 with the exact `detail` messages in D.4. Also 401, 403, 404 `exam_not_found`.

Action	Allowed when	Effect	Refusal code
publish	Draft, `startsAt` in the future, at least one question, every question valid under the current rules and the current `Exams:MaxMcqOptions`	status Published, `publishedAt` = server time	`exam_not_publishable`
unpublish	Published, before `startsAt` (attempt check deferred to Phase 4)	status Draft, `publishedAt` = null	`exam_not_unpublishable`
cancel	Published, before `startsAt`	status Cancelled (terminal), `publishedAt` kept	`exam_not_cancellable`

C.5 Question endpoints (Step 2.3)

Question response `QuestionResponse` (exact fields and order)

```
{
  "id": "<question guid>",
  "examId": "<exam guid>",
  "type": "Mcq",                       // "Mcq" | "Essay"
  "text": "Capital of France?",
  "marks": 2.50,                       // number, always 2 decimals
  "modelAnswer": "Paris is the capital.",   // string | null
  "position": 1,                       // 1..n, assigned by the server
  "options": [                         // [] for Essay
    { "id": "<guid>", "text": "Paris", "isCorrect": true,  "position": 1 },
    { "id": "<guid>", "text": "Lyon",  "isCorrect": false, "position": 2 }
  ]
}
```

GET /api/exams/{examId}/questions

200 `{ "items": [QuestionResponse] }` in position order, **not paged**; allowed at any exam status or time. 404 `exam_not_found`.

POST /api/exams/{examId}/questions · PUT /api/questions/{id}

```
{
  "type": "Mcq",                    // "Mcq" | "Essay" (any letter case; numbers rejected)
  "text": "Capital of France?",
  "marks": 2.5,
  "modelAnswer": "Paris is the capital.",   // optional; blank becomes null
  "options": [
    { "text": "Paris", "isCorrect": true },
    { "text": "Lyon",  "isCorrect": false },
    { "text": "Nice" }              // missing isCorrect = false
  ]
}
```

POST: **201** `QuestionResponse`, appended at the end. PUT: **200**, full replacement: keeps id, exam and position; old options deleted, new options stored with **new ids** in the order sent. Writes allowed only for a Draft or Published

exam before its start → otherwise 409 `exam_not_editable`. 404: `exam_not_found` (POST), `question_not_found` (PUT; unknown, malformed or other instructor's question: "The question was not found.").

Validation messages (400, field keys `type`, `text`, `marks`, `modelAnswer`, `options`, `options[i].text`, `options[i]`, index from 0): "Type is required.", "Type must be Mcq or Essay.", "Text is required.", "Text must be at most 4000 characters.", "Marks are required.", "Marks must be greater than 0.", "Marks must be at most 9999.99.", "Marks can have at most 2 decimal places.", "Model answer must be at most 4000 characters.", "An MCQ question needs at least 2 options.", "An MCQ question can have at most N options." (N = `Exams:MaxMcqOptions`, default 8), "An MCQ question needs exactly one correct option.", "Options must not repeat the same text.", "Option is required.", "Option text is required.", "Option text must be at most 1000 characters.", "An Essay question has no options.".

DELETE /api/questions/{id}

204, empty body. Later questions move up one position. 409 `exam_not_editable` when the exam is not editable, or when deleting the **last** question of a Published exam ("A published exam must keep at least one question."). 404 `question_not_found`.

C.6 Admin endpoints (implemented; reference only – no React screens)

Policy `AdminOnly` (`AdminAccountsController`, route prefix `api/admin`). They manage Student and Instructor accounts only, never Admin accounts (D21). Admin screens belong to the Flutter app (owner decision). The only use in this project's frontend work is creating an instructor account for local testing (F.6).

Endpoint	Contract summary
`GET /api/admin/accounts?role=&status=&page=&pageSize=`	Students and instructors only; role Student/Instructor, status Active/Deactivated, page ≥ 1, pageSize 1–100 (default **20**). 200 `{ items, page, pageSize, totalCount }`; items have `id, fullName, email, role, accountStatus, mustChangePassword, temporaryPasswordExpiresAt, isLockedOut, createdAt, lastLoginAt, statusChangedAt, statusReason`. `Cache-Control: no-store`.
`GET /api/admin/accounts/{id}`	One account with the same 12 fields; 404 `account_not_found`.
`POST /api/admin/instructors`	`{ fullName, email }` → 201 `{ id, fullName, email, role, temporaryPassword, temporaryPasswordExpiresAt }` (password shown only once); 409 `duplicate_account`.
`POST /api/admin/accounts/{id}/temporary-password`	200 `{ temporaryPassword, temporaryPasswordExpiresAt }`.
`POST /api/admin/accounts/{id}/deactivate` · `/reactivate`	`{ reason }` (required for deactivate, optional for reactivate, max 500) → 204; 409 `account_status_unchanged`.

D. Business logic and UI behaviour

D.1 Exam creation and editing

- Fields: title, description, start date/time, duration. Never show inputs for status, warning limit, published time or owner.
- Start time must be in the future (server clock). On edit, the check applies only if the start time is changed; send the original `startsAt` string back unchanged if the user did not touch the field (avoids minute-rounding being treated as a change).
- Edit is possible only while `status` is Draft or Published *and* `phase` is Upcoming. Hide or disable "Edit" otherwise and explain why. The server is authoritative: if the exam starts while the page is open, PUT returns 409 → show `detail` and reload the exam.
- Concurrent edits: the later save wins (no version field). **RECOMMENDATION** reload after saving.

D.2 Time zone handling

- Input: the instructor enters local wall-clock time (`<input type="datetime-local">`). Send it with the browser's offset, e.g. `2026-11-01T09:00:00+03:00`, or as UTC `Z` via `Date.toISOString()`. Both are valid. Never send a value without an offset (rejected) and never treat local time as UTC.
- Output: the server always returns UTC (`+00:00`). Display in local time and state the time zone near date fields.
- `phase` is computed by the server when the exam is read; it can become stale while a page stays open.

D.3 Duration

Whole minutes, 1–360 (6 hours). `endsAt` is computed by the server.

D.4 States and lifecycle actions

Exam	Edit exam / questions	Delete	Publish	Unpublish	Cancel
Draft, before start	allowed	204	allowed if valid	409 "The exam is not published."	409 "Only a published exam can be cancelled."
Draft, at/after start	409 `exam_not_editable`	204	409 "The start time has passed."	409 "The exam is not published."	409 "Only a published exam can be cancelled."
Published, before start	allowed (not the last question's deletion)	409 `exam_not_editable`	409 "The exam is already published."	allowed	allowed
Published, at/after start	409 `exam_not_editable`	409	409 "The exam is already published."	409 "The start time has passed."	409 "The start time has passed."
Cancelled (terminal)	409 `exam_not_editable`	409	409 "A cancelled exam cannot be published."	409 "A cancelled exam cannot be unpublished."	409 "The exam is already cancelled."

Publish refusal messages (first blocking problem, questions in position order): "The exam has no questions.", "Question N has no text.", "Question N has more than 4000 characters of text.", "Question N has invalid marks.", "Question N has a model answer longer than 4000 characters.", "Question N is an essay question and must not have options.", "Question N has an option without text.", "Question N has an option longer than 1000 characters.",

"Question N needs at least 2 options.", "Question N has more options than the current maximum of M.", "Question N needs exactly one correct option.", "Question N repeats an option text.". In practice, for API-created questions only the first, the start-time message and the maximum-options message can occur.

UI rules: show Publish only for Draft; Unpublish and Cancel only for Published; Delete only for Draft; disable lifecycle buttons when `phase` is not Upcoming (Delete stays available for a Draft). Every action opens a confirmation dialog; Cancel and Delete are marked **irreversible**. On success, replace the displayed exam with the 200 response. On 409/404, show `detail` (or the friendly 404 text) and reload the exam. Never change the status locally without a server response.

Known backend limitation: under a narrow race (another request changes the status at the same moment) a refusal's `detail` can read "The start time has passed." although the real reason was the status. The 409 itself is correct.

D.5 Questions

- Whole-question writes: the form sends the full question including all options; a PUT replaces everything.
- Positions are server-assigned (append; renumber after delete). There is no reordering; don't offer drag-and-drop.
- MCQ: at least 2 options; exactly one correct (use radio buttons); option texts must be distinct (trimmed, case-insensitive); the maximum count is a server setting (default 8) that is **not exposed by the API** – don't hard-block at 8; show the server's 400 message if exceeded. If the maximum was lowered, existing over-limit questions stay stored; editing *another* question still works; publishing reports the over-limit question.
- Essay: no options (send `[]`); hide option controls.
- Marks: number > 0, ≤ 9999.99, at most 2 decimals; display with 2 decimals.
- Model answer: optional; visible to the instructor only (students never receive it).
- Writes allowed only while the exam is Draft/Published and Upcoming. Questions can always be read.
- Deleting the only question of a Published exam is refused (409); explain it in the delete dialog.

D.6 Validation errors

Map `errors` keys to fields: sign-up keys `fullName`, `email`, `password`, `confirmPassword`; exam keys `title`, `description`, `startsAt`, `durationMinutes`; question keys `type`, `text`, `marks`, `modelAnswer`, `options` (group), `options[i].text` / `options[i]` (the i-th option in the order sent, 0-based); auth keys `email`, `password`, `currentPassword`, `newPassword`. Show `detail` above the form. Malformed-JSON errors use framework key names – show the general message.

D.7 Pagination, sorting, filters

Exams: `page` / `pageSize` (1–100, default 10) and `status` only. No search, no custom sort (newest start first is fixed). Questions: no paging. **RECOMMENDATION** keep filter and page in the URL query string.

D.8 Ownership and access denied

Instructors only see their own exams. Another instructor's exam id gives 404 – show "not found or not yours". Admin and Student accounts get 403 without a code on all exam endpoints – show an "instructor access only" screen.

D.9 Temporary password flow

Admin-created instructors receive a temporary password (default lifetime 72 hours, `Auth:TemporaryPasswordHours`). Login returns `mustChangePassword: true`; route the user to the change-password page; all other endpoints answer 403 `password_change_required` until changed. An expired temporary password → 403 `temporary_password_expired` at login: tell the user to ask an administrator.

D.10 Student sign-up (owner decision: included)

- Form fields exactly as C.3: full name, email, password, confirm password. No role, university or any other field.
- Client checks mirror the server (required fields, 150-character name, basic email shape, password policy, confirmation matches); the server stays authoritative and its `errors` are mapped to the four fields (`password` may carry several Identity policy messages).
- On 201: clear the form (including both passwords) and show a success panel: "Your student account has been created. Student exam features are not available in this app yet." with a "Sign in" link. Do **not** sign in automatically and do not store anything from the response.
- On 409 `duplicate_account`: show the message on the email field. On 429: "Too many attempts, wait a minute".
- A Student who signs in gets the "Instructor access only" screen plus the same note; exam lists for students do not exist (`me/exams` is blocked by §2.4).

E. Authentication and security

Topic	Verified behaviour and required frontend handling
Sign-in	`POST /api/auth/login` → `TokenResponse`. No cookies are used; tokens are only in the JSON body.
Access token	JWT, HS256, issuer `ExamProctoring.Api`, audience `ExamProctoring.Clients`, lifetime 15 minutes (configurable), 30 s clock skew. Send `Authorization: Bearer <token>`. Use `accessTokenExpiresAt` to refresh slightly early. Claims (`sub`, `role`, security stamp) should not be parsed for authorization; use `user` from the token response.
Per-request account gate	The server checks on every request that the account exists, is Active and has the same security stamp. After a password change or admin action, old access tokens get 401 `token_invalid`.
Refresh	`POST /api/auth/refresh` with the refresh token → a new pair. Refresh tokens rotate; reusing an old one revokes the session (401 `refresh_token_reused`). **Required:** only one refresh at a time across the whole app (single-flight); retry the original request once after a successful refresh; sign out when refresh fails with 401/403.
Logout	`POST /api/auth/logout` with the refresh token and the access token; 204. Clear local tokens even if the call fails. The access token remains technically valid until expiry (max 15 min).
Protected routes	Require a session; redirect to the change-password page while `mustChangePassword`; show "instructor access only" for other roles. Frontend checks are for UX only.
401 vs 403	401 `token_invalid`: try one refresh, else sign in again. 403 `password_change_required`: change password. 403 without code: wrong role. Other 401/403 codes at login: show `detail`.
Token storage	**Required for this project** (as already implemented in the draft's `src/auth/tokenStore.ts`): the access token in memory only; the refresh token in `sessionStorage`, so a reload in the same tab keeps the session and closing the tab ends it. **Never** store either token in `localStorage`, URLs or logs, and never render untrusted HTML. Trade-off: any script running on the page can read `sessionStorage`, so XSS must be prevented (React escapes text; no `dangerouslySetInnerHTML`). Keeping the refresh token in memory only is stricter but loses the session on reload. HttpOnly cookies would be safer still, but the backend returns tokens only in JSON (no cookie option) and changing that is a backend change. The only value this app keeps in `localStorage` is the language preference (H.9).
Rate limits	Per IP: login 10/min, refresh 30/min, register 5/min → 429.
Secrets	No backend secret belongs in the frontend. Everything in `VITE_*` variables is public.

> **Security limitations to document in the frontend README:** tokens readable by JavaScript (no HttpOnly cookies); no rate limits or body-size limits on exam/question endpoints (backend hardening planned for Phase 7); no CSRF concerns because no cookies are used.

F. Frontend-to-backend integration

F.1 Backend URLs (verified in `src/ExamProctoring.Api/Properties/launchSettings.json`)

Launch profile	URLs	Environment
`https`	`https://localhost:7046` and `http://localhost:5168`	Development
`http`	`http://localhost:5168`	Development

Both profiles bind to **localhost only**. `Program.cs` runs `UseHttpsRedirection` before CORS, and its effect depends on the profile:

- `dotnet run --project src/ExamProctoring.Api --launch-profile http`: no HTTPS endpoint, so nothing is redirected; `http://localhost:5168` answers directly (verified live 2026-10-23). This is the profile the owner uses and the **default for this project**: no development certificate is involved.
- `--launch-profile https`: requests to `http://localhost:5168` are redirected to `https://localhost:7046`; a cross-origin browser call to the http URL would then fail its CORS preflight, so use the https URL with this profile.

Running the published DLL without a profile uses the Production environment (no User Secrets, no Swagger).

F.2 CORS (verified in `Program.cs` and `appsettings.json`)

One policy allows only the origins in `Cors:WebExamOrigins` (any header, any method, no credentials). The list is **empty**, so a browser app on another origin is blocked. Two options:

1. **RECOMMENDED for development** Vite dev-server proxy: forward `/api` to the backend URL of the profile in use (`http://localhost:5168` for the `http` profile, `https://localhost:7046` for `https`); the browser stays same-origin; no backend change.
2. Direct calls: the backend team adds the frontend origin (e.g. `http://localhost:5173` in development) to `Cors:WebExamOrigins`. This is a backend configuration change (decision D24) requiring approval; production needs the real origin.

F.3 HTTPS certificate

Only relevant with the `https` profile. The ASP.NET Core development certificate (`dotnet dev-certs https --trust`) is trusted by Windows browsers but not by Node.js, so a Vite proxy to `https://localhost:7046` needs `secure: false` (the draft's default, local development only) or Node configured to trust the certificate. With the `http` profile and a proxy to `http://localhost:5168` no certificate is involved.

F.4 Environment variables

```
# .env.example (placeholders only – no secrets)
VITE_API_BASE_URL=                          # empty = same origin (dev proxy); or the API URL when CORS
allows it
VITE_DEV_PROXY_TARGET=http://localhost:5168   # "http" profile; use https://localhost:7046 with the "https"
profile
VITE_DEV_PROXY_VERIFY_TLS=false               # only matters for an https target
```

The draft's current `frontend/.env.example` sets `VITE_DEV_PROXY_TARGET=https://localhost:7046`; change it to match the profile actually used. `.env.local` is ignored by Git.

F.5 API client RECOMMENDATION

- One `apiRequest<T>(path, { method, body, signal, auth })` function: JSON in/out, Bearer header, 204 → undefined, ProblemDetails → typed `ApiError { status, code, detail, fieldErrors, kind: 'http' | 'network' | 'aborted' }`.
- Refresh handling inside the client (single-flight, retry once, sign out on failure); never refresh after login failures (codes other than `token_invalid`).
- Pass an `AbortController` signal from page loaders to cancel obsolete requests.
- Never show `detail` of 5xx responses (in Development it may contain internal exception text).

```
// src/api/exams.ts – example service functions matching the real routes
export const listExams = (p: { status?: ExamStatus; page: number; pageSize: number }, signal?: AbortSignal)
=> {
  const q = new URLSearchParams({ page: String(p.page), pageSize: String(p.pageSize) });
  if (p.status) q.set('status', p.status);
  return apiRequest<ExamPage>(`/api/exams?${q}`, { signal });
};
export const getExam    = (id: string, signal?: AbortSignal) => apiRequest<Exam>
(`/api/exams/${encodeURIComponent(id)}`, { signal });
export const createExam = (r: ExamRequest) => apiRequest<Exam>('/api/exams', { method: 'POST', body: r });
export const updateExam = (id: string, r: ExamRequest) => apiRequest<Exam>
(`/api/exams/${encodeURIComponent(id)}`, { method: 'PUT', body: r });
export const deleteExam = (id: string) => apiRequest<void>(`/api/exams/${encodeURIComponent(id)}`, { method:
'DELETE' });
export const lifecycle  = (id: string, action: 'publish' | 'unpublish' | 'cancel') =>
  apiRequest<Exam>(`/api/exams/${encodeURIComponent(id)}/${action}`, { method: 'POST' });

// src/api/questions.ts
export const listQuestions  = (examId: string, signal?: AbortSignal) => apiRequest<{ items: Question[] }>
(`/api/exams/${examId}/questions`, { signal });
export const createQuestion = (examId: string, r: QuestionRequest) => apiRequest<Question>
(`/api/exams/${examId}/questions`, { method: 'POST', body: r });
export const updateQuestion = (id: string, r: QuestionRequest) => apiRequest<Question>
(`/api/questions/${id}`, { method: 'PUT', body: r });
export const deleteQuestion = (id: string) => apiRequest<void>(`/api/questions/${id}`, { method: 'DELETE'
});
```

```
// Local datetime-local value -> API value with an explicit offset
export function localInputToApi(value: string): string {
  const d = new Date(value);                      // "2026-11-01T09:00" is parsed as LOCAL time
  const off = -d.getTimezoneOffset(), sign = off >= 0 ? '+' : '-', a = Math.abs(off);
  const p = (n: number) => String(n).padStart(2, '0');
  return `${d.getFullYear()}-${p(d.getMonth() +
1)}-${p(d.getDate())}T${p(d.getHours())}:${p(d.getMinutes())}:00` +
         `${sign}${p(Math.floor(a / 60))}:${p(a % 60)}`;
}
```

F.6 Backend setup for live integration (each developer machine)

The owner's machine is already set up (database migrated, first Admin seeded on 2026-10-23, API running with the `http` profile). A developer running the backend on their own machine needs:

1. **.NET SDK 10.0.402** (pinned in `global.json`) and a reachable **SQL Server**.
2. **User Secrets** for `src/ExamProctoring.Api` (never in `appsettings*.json`, never committed):

```
dotnet user-secrets set "ConnectionStrings:Default" "<your SQL Server connection string>" --project
src/ExamProctoring.Api
dotnet user-secrets set "Jwt:SigningKey" "<Base64 of at least 32 random bytes>" --project
src/ExamProctoring.Api
dotnet user-secrets set "Seed:AdminEmail" "<first admin email>" --project src/ExamProctoring.Api
dotnet user-secrets set "Seed:AdminPassword" "<first admin password>" --project src/ExamProctoring.Api
```

A missing or short `Jwt:SigningKey` stops the API at startup (`JwtOptions` validation). The seed settings are read **only while no Admin exists** (D20): on an empty database the API stops without them; once the Admin exists, changing them changes nothing. They must satisfy the password policy (8+ characters, upper case, lower case, digit). Prefer ASCII letters for passwords typed from PowerShell.

3. **Migrations**, applied manually (never at startup): `dotnet tool restore`, then `dotnet ef database update --project src/ExamProctoring.Infrastructure --startup-project src/ExamProctoring.Api`.
4. **Start the API**: `dotnet run --project src/ExamProctoring.Api --launch-profile http` → `http://localhost:5168`; check `http://localhost:5168/health`.
5. **Create an instructor account** (no React admin screens; call the admin API once from Windows PowerShell). Passwords are typed at hidden prompts and the JSON is sent as UTF-8; the instructor's temporary password is printed once, so treat the output as secret:

```
$base = 'http://localhost:5168'
function Plain($s) {
[Runtime.InteropServices.Marshal]::PtrToStringBSTR([Runtime.InteropServices.Marshal]::SecureStringToBSTR
($s)) }
function Utf8Json($o) { ,[Text.Encoding]::UTF8.GetBytes(($o | ConvertTo-Json)) }
$ct = 'application/json; charset=utf-8'
$adminEmail = Read-Host 'Admin email'
$adminPass  = Read-Host 'Admin password' -AsSecureString
$login = Invoke-RestMethod -Method Post -Uri "$base/api/auth/login" -ContentType $ct -Body (Utf8Json @{
email = $adminEmail; password = (Plain $adminPass) })
$name  = Read-Host 'Instructor full name'
$email = Read-Host 'Instructor email'
$created = Invoke-RestMethod -Method Post -Uri "$base/api/admin/instructors" -ContentType $ct -Headers
@{ Authorization = "Bearer $($login.accessToken)" } -Body (Utf8Json @{ fullName = $name; email = $email
})
"Temporary password (shown once): $($created.temporaryPassword)  expires:
$($created.temporaryPasswordExpiresAt)"
Remove-Variable adminPass, login, created
```

The instructor then signs in to the React app with that temporary password (valid 72 hours) and must change it (D.9). Five wrong passwords lock an account for 15 minutes.

6. **Node.js** 22.12 or later (24 LTS recommended) on the frontend machine.

F.7 Access from another computer on the same network

`localhost` always means "this computer". A browser on another computer must use the LAN address of the machine that runs the servers (for example `http://192.168.1.20:5173`; find it with `ipconfig`). The API's launch profiles listen on localhost only and `Cors:WebExamOrigins` is empty.

- **RECOMMENDED** Run the Vite dev server on the **same machine as the API** with `npm run dev -- --host` (listens on all interfaces). Other computers open `http://<LAN-IP>:5173`; the Vite proxy forwards `/api` to `http://localhost:5168` on that machine. No backend binding or CORS change is needed; allow inbound TCP 5173 in Windows Firewall (private network profile only).
- Alternative (backend change, needs the owner's approval; **not done**): make the API listen on the LAN (for example `--urls http://0.0.0.0:5168` or a launch-profile change), allow inbound TCP 5168 in the firewall, and add the frontend's exact origin (for example `http://192.168.1.21:5173`) to `Cors:WebExamOrigins`; then set `VITE_API_BASE_URL` to `http://<API-LAN-IP>:5168`.

- Plain HTTP on a LAN sends passwords and tokens unencrypted; acceptable only on a trusted development network.

G. Recommended React project structure

Stack (already chosen by the existing draft, which is to be reused): Vite + React + TypeScript, React Router, plain CSS, Vitest + Testing Library + jsdom. Location: `frontend/` at the repository root (separate from `src/`, the .NET backend). The tree below is the target; the draft's actual layout (G.2) may be kept where it differs only in folder names. For translations, a small in-house dictionary (G.3) is enough; a library such as `i18next` is optional.

G.1 Target structure

```
frontend/
  index.html  package.json  tsconfig.json  vite.config.ts  .env.example  README.md
  src/
    app/          App.tsx (providers), routes.tsx
    api/          client.ts (apiRequest, refresh), errors.ts (ApiError, describeError),
                  auth.ts, exams.ts, questions.ts
    types/        api.ts – DTO interfaces mirroring the backend (below)
    features/
      auth/       AuthContext.tsx, tokenStore.ts, RequireInstructor.tsx, LoginPage.tsx,
ChangePasswordPage.tsx,
                  RegisterPage.tsx (student sign-up)
    i18n/         LanguageContext.tsx (lang + dir), ar.ts, en.ts (same keys), LanguageToggle.tsx
      exams/      ExamListPage.tsx, ExamDetailsPage.tsx, ExamForm.tsx, ExamActions.tsx,
Create/EditExamPage.tsx
      questions/  QuestionsPanel.tsx, QuestionForm.tsx
    components/   ConfirmDialog, Alert, Spinner, EmptyState, FormField, Pagination, StatusBadge
    layouts/      AppLayout.tsx (header + outlet)
    hooks/        useLoader.ts (abortable loading)
    utils/        dates.ts (offset conversion, formatting), validation.ts (early checks)
    test/         setup.ts, fixtures.ts
```

DTO interfaces to define (must match the backend exactly)

```
type Role = 'Admin' | 'Instructor' | 'Student';
interface AuthUser { id: string; fullName: string; email: string; role: Role }
interface TokenResponse { tokenType: string; accessToken: string; accessTokenExpiresAt: string;
refreshToken: string;
  refreshTokenExpiresAt: string; mustChangePassword: boolean; user: AuthUser }
type ExamStatus = 'Draft' | 'Published' | 'Cancelled';
type ExamPhase = 'Upcoming' | 'Open' | 'Closed';
interface Exam { id: string; title: string; description: string | null; startsAt: string; durationMinutes:
number;
  endsAt: string; status: ExamStatus; phase: ExamPhase; warningLimit: number; publishedAt: string | null }
interface ExamPage { items: Exam[]; page: number; pageSize: number; totalCount: number }
interface ExamRequest { title: string; description: string | null; startsAt: string; durationMinutes: number
}
type QuestionType = 'Mcq' | 'Essay';
interface QuestionOption { id: string; text: string; isCorrect: boolean; position: number }
interface Question { id: string; examId: string; type: QuestionType; text: string; marks: number;
  modelAnswer: string | null; position: number; options: QuestionOption[] }
interface QuestionRequest { type: QuestionType; text: string; marks: number; modelAnswer: string | null;
  options: { text: string; isCorrect: boolean }[] }
interface ProblemDetails { status?: number; title?: string; detail?: string; code?: string;
  errors?: Record<string, string[]> }
interface RegisterRequest { fullName: string; email: string; password: string; confirmPassword: string }
interface RegisterResponse { id: string; fullName: string; email: string; role: 'Student' }
```

Backend-imposed requirements (not optional): string enums with exact casing in responses, offset-bearing `startsAt`, camelCase JSON, Bearer header, ProblemDetails parsing. Everything else in this section is a recommendation.

G.2 Reusing the existing `frontend/` draft (owner decision)

The draft (36 source files, untracked) was written against the contracts in Section C, but has **never been installed, type-checked, tested or built**. Reuse it; do not replace it. First run `npm install`, `npm run typecheck`, `npm test` and `npm run build`, fix what fails, and only then extend it.

Draft files	Status	Notes
`src/api/client.ts`, `errors.ts`, `auth.ts`, `exams.ts`, `questions.ts`	Reuse	Single-flight refresh, `ApiError`/ProblemDetails parsing, routes as in C. Add `register()` to `auth.ts`.
`src/types/api.ts`	Reuse	Add the register types (G.1).
`src/auth/tokenStore.ts`, `AuthContext.tsx`, `ProtectedRoute.tsx`	Reuse	Already follows E (memory + `sessionStorage`, never `localStorage`).
`src/utils/dates.ts`, `validation.ts`, `hooks/useLoader.ts`	Reuse	Date formatting must take the active language (H.9).
`src/components/*` (10 files), `src/pages/*` (5 files), `App.tsx`, `main.tsx`	Reuse, then translate	All UI text is hard-coded English; move it to the dictionaries.
`src/styles.css`	Adapt	4 rules use physical left/right properties; replace them with logical ones for RTL (H.9).
`index.html`	Adapt	Fixed `lang="en"`; set `lang`/`dir` at runtime.
4 test files, `src/test/*`	Reuse	Extend per K.
`.env.example`	Adapt	Proxy target per F.4.
`package.json`	**UNVERIFIED**	Pinned versions (React 19.3.1, react-router-dom 7.18.4, Vite 8.3.5, TypeScript 7.0.2, Vitest 5.0.5, jsdom 30.1.3 and others) have never been installed; no lockfile. Adjust any version that does not resolve and record the change.
Missing	Build	Sign-up page and route, localization layer, language toggle.

G.3 Translation dictionaries

One dictionary per language with identical keys (TypeScript should fail if a key is missing in either). Keys for every visible string, including validation messages the frontend generates and friendly texts per error `code` (Section I). Exam titles, descriptions, question text and option text are user content and are shown exactly as stored, in either language.

H. UI/UX and validation specification

H.1 Navigation and layout

Top header (app name, Exams, New exam, user name, Sign out); content max-width ~1100 px; on phones the header wraps and tables become cards. Pages: Sign in and Change password are centred cards without the header.

H.2 Exam list

Columns: Title (link; description excerpt), Starts (local), Duration, Status badge (Draft grey, Published green, Cancelled red), Phase badge (Upcoming / In progress / Ended). Toolbar: Status select, Per-page select (values within 1–100). Pagination: Previous/Next, "Page x of y", "x–y of total". States: loading spinner, error with "Try again", empty ("No exams yet" with create link; "No draft exams" when filtered; "past the end" when page > last).

H.3 Exam form (create/edit)

Field	Control	Client checks (mirror backend)
Title	text	required after trim; ≤ 200
Description	textarea + counter	optional; ≤ 2000 after trim; blank → null
Start	datetime-local + time-zone hint	required; valid; in the future (only if changed on edit); send with offset
Duration (min)	number, step 1	required; integer 1–360

Show warning limit read-only on edit. Buttons: Cancel (back), Save/Create (disabled while saving, label "Saving…"). On server 400 map field errors; on 409 show `detail`.

H.4 Exam details and actions

Details grid (Starts, Ends, Duration, Published, Warning limit). Action buttons per D.4; disabled buttons have an explanatory note ("This exam has started …"). Confirmation dialogs: Publish ("the server checks every question"), Unpublish, Cancel (**cannot be undone**, danger style), Delete (**deletes the exam and all questions permanently**, danger style). Success alerts: "The exam is published." / "back in Draft" / "cancelled"; Delete navigates to the list.

H.5 Questions panel

Header with count and total marks; "Add question" when editable; read-only notice otherwise; list items show Q number, MCQ/Essay badge, marks, text, options (correct marked with text "(correct answer)" for screen readers, not colour alone), model answer; Edit/Delete per item. Empty state: "Add at least one question before publishing." Delete dialog mentions renumbering and the Published-exam rule.

H.6 Question editor

Field	Control	Client checks
Type	select MCQ / Essay	Essay hides options and sends `[]`
Marks	decimal text input	required; number > 0; ≤ 9999.99; ≤ 2 decimals
Text	textarea	required after trim; ≤ 4000
Options (MCQ)	rows: radio (correct) + text + Remove; "Add option"	≥ 2 rows (Remove disabled at 2); each text required ≤ 1000; exactly one radio; no repeated text (trim, case-insensitive); no fixed maximum in the UI (server reports it)

Model answer	textarea	optional; ≤ 4000; blank → null

H.7 Sign-in and password forms

Login: email (required), password (required), link "Create a student account". Change password: current, new, repeat (must match; frontend-only); hint with the password policy; server messages per field. Sign up: full name, email, password, confirm password (all required; policy hint under the password; the confirm field is sent as `confirmPassword`, it is part of the backend contract); success panel per D.10.

H.8 Accessibility and responsiveness

- Labels linked to inputs; errors linked with `aria-describedby`; `aria-invalid` on invalid fields; alerts with `role="alert"`; dialogs with `role="dialog"`, focus on open, Escape to close.
- Visible focus outlines; colour is never the only signal; respect reduced motion.
- Breakpoints: phone (single column, cards), tablet/desktop (two-column forms, table).

H.9 Arabic / English, RTL / LTR (owner decision)

Topic	Rule
Languages	Arabic (`ar`, right-to-left) and English (`en`, left-to-right). **Arabic is the default** on a first visit.
Toggle	Visible on every page (header; also the sign-in, sign-up and change-password cards). Labels in their own language: "العربية" and "English". Switching is instant, keeps the current page and form input, and does not touch the session.
Persistence	Store only the chosen language code in `localStorage` (for example key `lang`), wrapped in try/catch; fall back to Arabic. No tokens or personal data there.
Document direction	Set `<html lang="ar" dir="rtl">` or `<html lang="en" dir="ltr">` whenever the language changes.
CSS	Use logical properties (`margin-inline-start`, `padding-inline-end`, `inset-inline-start`, `text-align: start`) instead of left/right; mirror direction-bearing icons (arrows, "previous/next"); do not mirror numbers, code, emails or the clock.
Mixed text	Emails, GUIDs, times and English server messages inside Arabic text are wrapped with `dir="ltr"` or `<bdi>`; user-entered exam and question text uses `dir="auto"`.
Dates and numbers	Format with `Intl.DateTimeFormat` / `Intl.NumberFormat` using the active language (`ar` or `en`) and the browser's time zone; show the time-zone name next to date fields. Inputs (`datetime-local`, marks, duration) accept Western digits as the backend expects.
Server messages	The backend returns English `detail` and field messages. Show a translated text for known `code` values (Section I) and known field rules; when the server sends a message the app cannot map (for example a publish refusal naming question N), show the server's English text as is, marked `dir="ltr"`.
Fonts	A font stack that covers Arabic (for example `"Segoe UI", Tahoma, "Noto Naskh Arabic", sans-serif`).

Out of scope / pending: webcam monitoring, audio detection, suspicious-behaviour dashboards, AI question generation, student exam-taking pages, results, integrity and collusion reports.

I. Error handling matrix

Codes below are verified in `ErrorCodes.cs` / `ErrorResults.cs` and the auth handlers. "UI behaviour" is a **RECOMMENDATION**.

HTTP	code	Meaning (backend)	UI behaviour
400	`validation_failed`	Field rules failed; `errors` lists messages per field	Show messages next to fields; general `detail` above the form
401	`token_invalid`	Missing/invalid/expired token, deactivated account, changed security stamp; or refresh token invalid/expired	Refresh once and retry; otherwise sign out → login ("Your session has ended")
401	`invalid_credentials`	Wrong email or password	Show `detail` on the login form
401	`account_locked`	Too many failed attempts (15 min)	Show `detail`; no retry loop
401	`refresh_token_reused`	Old refresh token reused; session revoked	Sign out; explain the session ended
403	`password_change_required`	Temporary password must be changed	Go to the change-password page
403	`account_deactivated`	Account deactivated by an admin	Show `detail`; sign out
403	`temporary_password_expired`	Temporary password expired	Show `detail` ("ask an administrator")
403	(none)	Wrong role	"Instructor access only" screen
404	`exam_not_found`	Unknown, malformed or another instructor's exam	"Exam not found or not yours"; link back to the list
404	`question_not_found`	Unknown, malformed or another instructor's question	Show message; reload the question list
409	`exam_not_editable`	Exam started/cancelled, Published/Cancelled delete, or last question of a Published exam	Show `detail`; reload exam
409	`exam_not_publishable`	Publish refused (first blocking problem)	Show `detail`; reload exam
409	`exam_not_unpublishable`	Unpublish refused	Show `detail`; reload exam
409	`exam_not_cancellable`	Cancel refused	Show `detail`; reload exam
409	`duplicate_account`	Sign-up: an account with this email already exists	Translated message on the email field
409	`account_status_unchanged`	Admin only (no React screens)	–
404	`account_not_found`	Admin only (no React screens)	–
413	(none)	Auth request body > 16 KB	Generic message
429	(none)	Rate limit on login (10/min), refresh (30/min), register (5/min) per IP	"Too many attempts, wait a minute"

500	(none)	Unhandled server error (no stack trace; message only in Development)	Generic message; never show `detail`
–	network	No response (server down, CORS block, certificate)	"Cannot reach the server"; retry button

J. Implementation plan

#	Phase	Tasks	Depends on	Acceptance criteria
1	Contract verification	Re-read the controllers/DTOs listed in Section 0 and compare with Section C (scope is fixed by the owner decisions in Section 0)	–	Differences recorded (source wins)
2	Validate the existing draft	With Node 22.12+ installed: `npm install`, `npm run typecheck`, `npm test`, `npm run build` in `frontend/`; fix failures (including unresolvable package versions); set `.env.example` per F.4	1	All four commands pass on the draft as it is
3	API client + auth (review)	Review the draft's `apiRequest`, `ApiError`, DTO types, token store and single-flight refresh against E; add `register()`	2	Unit tests for parsing, refresh, sign-out on failure, register
3a	Localization layer	Language context, `ar` / `en` dictionaries, toggle, `lang` / `dir` on `<html>`, logical CSS, locale-aware dates (H.9); move all existing text into the dictionaries	2	Every page renders correctly in Arabic (RTL, default) and English (LTR)
3b	Student sign-up	Register page, route, link from sign-in, success panel (D.10)	3, 3a	Field errors, 409 and 429 handled; no automatic sign-in
4	Routing + layout	Protected routes, password-change redirect, role screen, header	3	Route tests pass
5	Exam list	Status filter, pagination, states	4	Matches C.4 query rules
6	Exam create/edit/details	Forms, time-zone conversion, details page	5	Offset sent; UTC displayed locally; field errors shown
7	Questions	Panel, editor, MCQ/Essay, delete	6	Rules H.6; server errors per option
8	Lifecycle	Publish/unpublish/cancel/delete with dialogs	6	Table D.4 respected; 409 `detail` shown; exam reloaded
9	Validation + errors	Matrix I everywhere; network errors	3–8	No raw errors shown
10	Styling + a11y	Responsive layouts, focus, ARIA	5–8	Usable at 375 px and 1280 px; keyboard-only flow works
11	Tests + integration	Component tests; live run against a seeded local backend	All; F.6	Checklist K passes; live results reported honestly

K. Testing and acceptance checklist

Automated (mocked `fetch`) tests can run without the backend. Items marked **[LIVE]** need the backend running with a seeded admin, an instructor account, migrations applied and the dev proxy or CORS configured (F.6).

Authentication

☐ Login success stores the session; instructor lands on the exam list.

☐ Login 401 `invalid_credentials` / `account_locked`, 403 `account_deactivated` / `temporary_password_expired`, 429 show the right message and do not trigger a refresh.

☐ `mustChangePassword: true` forces the change-password page; other pages redirect there; success returns to the exam list. [LIVE]

☐ Expired access token → one refresh → original request retried; parallel requests trigger only one refresh.

☐ Refresh rejected → signed out, redirected to login.

☐ Logout calls the API and clears tokens even when the call fails.

☐ Student/Admin account sees "instructor access only"; backend 403 confirmed. [LIVE]

Exams

☐ List: default page 10; status filter in each value; page size change; next/previous; past-the-end page; empty state.

☐ Create: valid exam → 201 → details page; start time sent with the browser offset; displayed correctly in local time. [LIVE for round trip]

☐ Invalid submissions: empty title, 201-character title, 2001-character description, past start, duration 0 and 361 → messages; server 400 field errors mapped.

☐ Edit: unchanged start time is sent unchanged; started or cancelled exam shows read-only explanation; 409 from server shown.

☐ Delete Draft (confirm) → list; Delete not offered for Published/Cancelled.

☐ Unknown or another instructor's exam id → not-found message. [LIVE]

Questions

☐ MCQ create with 2+ options, one correct → appears at the end; Essay create sends no options.

☐ Client blocks: no text, marks 0 / 10000 / 1.005, < 2 options, no correct option, repeated option text.

☐ Server 400 for too many options shows the server message next to the options group. [LIVE]

☐ Edit replaces options; MCQ→Essay removes options; Essay→MCQ requires valid options.

☐ Delete renumbers positions; deleting the only question of a Published exam shows the 409 message. [LIVE]

☐ Read-only panel after start / when cancelled.

Lifecycle

☐ Publish with no questions → 409 "The exam has no questions." shown; exam reloaded.

☐ Publish valid Draft → status Published, publishedAt shown; Unpublish → Draft, publishedAt cleared; publish again → new time. [LIVE]

☐ Cancel requires confirmation; "Keep" sends nothing; confirm → Cancelled; all actions then hidden.

☐ Buttons disabled after the start time; Delete of a past Draft still possible.

Student sign-up

☐ Valid four-field form → 201 → success panel with the sign-in link; form and passwords cleared; no tokens stored; no automatic sign-in.

☐ Client blocks empty fields, a 151-character name and mismatched passwords; server 400 messages appear on `fullName`, `email`, `password` (several policy messages) and `confirmPassword`.

☐ An existing email → 409 `duplicate_account` shown on the email field. [LIVE]

☐ The request body contains exactly `fullName`, `email`, `password`, `confirmPassword`.

☐ A Student who signs in sees "Instructor access only" with the D.10 note. [LIVE]

Arabic / English

☐ First visit (empty storage) shows Arabic with `<html lang="ar" dir="rtl">`; the toggle switches to English with `lang="en" dir="ltr"` and back, keeping the page and form input.

☐ The choice survives a reload (stored language code only); if storage is unavailable the app still works in Arabic.

☐ Both dictionaries have the same keys (type check or test).

☐ Layouts mirror correctly in RTL (header, tables, forms, dialogs, pagination arrows); emails and times stay LTR.

☐ Dates and times are formatted for the active language in the browser's time zone; untranslatable server messages appear in English, LTR.

General

☐ Loading indicators and disabled buttons during requests; obsolete requests aborted on navigation.

☐ Network failure shows "Cannot reach the server"; 500 never shows server `detail`.

☐ Time zone: a value entered at 09:00 local is sent with the local offset and displayed back as 09:00 local.

☐ Responsive at 375 px, 768 px, 1280 px; keyboard and screen-reader basics.

L. Ready-to-use Claude Code prompt

Copy everything in the box into Claude Code after opening the repository root.

```
You are completing the React web frontend of the AI-Based Exam Proctoring System graduation project.
The backend is the ASP.NET Core (.NET 10) Web API in src/ (SQL Server, EF Core). The specification is
docs/AI_Exam_Proctoring_Frontend_Developer_Handover.pdf (editable source: the .html file beside it).

OWNER DECISIONS (fixed; do not reopen)
- Scope: the Instructor Dashboard, plus student sign-up. No admin screens (admin belongs to the Flutter
app).
- Reuse the existing frontend/ draft; do not replace it or start from scratch.
- Arabic and English with a language toggle; Arabic (RTL) is the default, English is LTR.
- Never stage, commit or push without the owner's explicit approval.

BEFORE CODING
1. Read the whole handover document, then inspect the repository: git status, frontend/ (an untracked draft
   that has never been installed or built), architecture-proposal.md (Revision 3.10: D11, D12, D25-D36, open
   decision 2.4). The handover document overrides architecture section 3.1 for the React scope.
2. Verify the backend contracts in source: src/ExamProctoring.Api/Controllers/*.cs,
   src/ExamProctoring.Application/{Auth,Exams}/*Request*.cs, *Response*.cs, *Validator*.cs,
   Auth/AccountFieldRules.cs, Auth/EmailRules.cs, Common/ErrorCodes.cs, Api/Errors/ErrorResults.cs,
   Api/Authorization/*, Program.cs, appsettings.json, Properties/launchSettings.json.
3. If the source differs from the document, the backend SOURCE wins. Record each difference. Never invent
   endpoints, fields, status codes or rules. If something needed does not exist, stop and report it as a
   blocker or ask me one concise question.
4. In frontend/: npm install, npm run typecheck, npm test, npm run build. Fix the draft until all pass
   before adding features. Report exact errors and any package version you had to change.

SCOPE (only these, all verified to exist)
- Auth: POST /api/auth/login, /refresh, /logout, /change-password; GET /api/auth/me.
  Access token in memory; refresh token in sessionStorage (never localStorage); single-flight refresh
  (refresh tokens rotate; reuse -> 401 refresh_token_reused revokes the session); retry once; sign out
  on failure; forced password change when mustChangePassword is true (other endpoints answer 403
  password_change_required).
- Student sign-up: POST /api/auth/register {fullName, email, password, confirmPassword} -> 201
  {id, fullName, email, role} with NO tokens. Rules and messages: handover C.3. No university field
  (D11/D12; the university decision was withdrawn). After 201 show the success panel with a sign-in link
  (handover D.10); no automatic sign-in. 409 duplicate_account on the email field; 429 rate limit.
- Exams: GET /api/exams?status=&page=&pageSize= (status Draft|Published|Cancelled, page>=1,
  pageSize 1-100 default 10, newest start first), POST /api/exams (201), GET/PUT/DELETE /api/exams/{id}
  (PUT 200, DELETE 204 Draft only). ExamRequest {title, description, startsAt, durationMinutes}:
  title 1-200, description <=2000, startsAt WITH an offset and in the future, duration 1-360.
  Never send warningLimit/status/publishedAt. Responses are UTC; show local time.
- Lifecycle: POST /api/exams/{id}/publish | /unpublish | /cancel -> 200 ExamResponse; refusals 409
  exam_not_publishable | exam_not_unpublishable | exam_not_cancellable with a readable detail.
  Cancel is terminal. Rules: handover section D.4.
- Questions: GET/POST /api/exams/{examId}/questions ({items}, not paged; 201), PUT/DELETE
  /api/questions/{id} (200/204). QuestionRequest {type: Mcq|Essay, text, marks, modelAnswer, options
  [{text, isCorrect}]}: text 1-4000, marks >0 <=9999.99 max 2 decimals, model answer optional <=4000,
  MCQ 2..N options (N is a server setting, default 8, not exposed), exactly one correct, no repeated
  text; Essay has no options. PUT replaces all options. Positions are server-assigned, no reordering.
  Writes only while the exam is Draft/Published and phase Upcoming; deleting the last question of a
  Published exam -> 409 exam_not_editable.
- OUT OF SCOPE: admin screens, student exam pages and exam taking, webcam/audio, proctoring, results,
  AI generation, reports (no endpoints exist for the student and proctoring features).

IMPLEMENTATION
- Keep the draft's structure (handover G.2); add src/i18n (language context, ar/en dictionaries with the
  same keys, toggle) and the register page; follow handover H.9 for RTL/LTR: lang and dir on <html>,
  logical CSS properties, locale-aware dates, only the language code in localStorage.
- Dev proxy /api -> http://localhost:5168 (launch profile "http"; use https://localhost:7046 only with the
  "https" profile). .env.example contains placeholders only, no secrets.
```

```
- Pages and states per sections B, D and H: loading, empty, validation (map errors keys such as startsAt,
  options[1].text, confirmPassword), 401/403/404/409/429/500 and network errors per section I with
  translated texts; confirmation dialogs for publish, unpublish, cancel (irreversible) and delete
  (irreversible). Never trust local status; use server responses and reload after 409/404. Never show
  detail of 5xx responses.
- Responsive and accessible (labels, aria-invalid, focus, keyboard) in both directions.

SAFETY
- Do not modify backend files (src/, tests/, migrations, appsettings, launchSettings) unless I authorize it.
- Preserve all existing work; do not overwrite or delete unrelated files.
- Never stage, commit, push, reset, stash or discard Git changes without my explicit authorization.
- Do not install system software without asking; npm packages inside frontend/ are fine.

VERIFY AND REPORT
- Run npm run typecheck, npm test and npm run build. Add tests per handover section K (API client,
  refresh, protected routes, sign-up, lifecycle errors, time zones, Arabic/English and RTL/LTR).
- Live integration needs the backend set up as in handover F.6 (User Secrets, migrations, an instructor
  account). If that is not available, say so; never claim untested integration works.
- Finish with: implemented features, created/changed files, commands run and their real results,
  remaining issues/blockers, and the exact commands to start backend and frontend.
```

End of document.